

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN II

Path Tracing Renderer with CUDA

NGUYỄN XUÂN KHÔI

MSSV: 20225022

khoi.nx225022@sis.hust.edu.vn

Giảng viên hướng dẫn: ThS. Nguyễn Duy Hiệp _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

TÓM TẮT NỘI DUNG ĐỒ ÁN

Hình ảnh được sinh từ máy tính đóng vai trò quan trọng trong nhiều ứng dụng của thế giới thực, từ lĩnh vực giải trí như điện ảnh, trò chơi điện tử đến các lĩnh vực kỹ thuật như kiến trúc và y học. Kết xuất dựa trên vật lý là lớp giải thuật mã hóa các định luật truyền ánh sáng thành mô hình toán học để tạo ra hình ảnh có độ chân thực cao. Trong số đó, Path Tracing là phương pháp tổng quát và chính xác, song có chi phí tính toán rất lớn do phải mô phỏng đường đi của hàng triệu tia sáng bằng tích phân Monte Carlo, dẫn tới thời gian kết xuất kéo dài và khó đạt tốc độ tương tác.

Trong đồ án này, tôi nghiên cứu và xây dựng một bộ kết xuất Path Tracing chạy song song trên kiến trúc CUDA của NVIDIA nhằm khai thác sức mạnh của GPU để đạt tốc độ kết xuất thời gian thực. Hệ thống hiện thực đầy đủ chu trình dò tia trên GPU, bao gồm: thuật toán Path Tracing với tích lũy theo thời gian và khử răng cưa; hệ thống vật liệu khuếch tán, kim loại và điện môi; mô hình camera thấu kính mỏng với hiệu ứng độ sâu trường ảnh và nhòe chuyển động; cơ chế cầu nối CUDA–OpenGL cho phép hiển thị trực tiếp từ bộ nhớ video; cùng bộ nạp cảnh theo định dạng Mitsuba 3 và giao diện điều khiển tương tác.

Kết quả cho thấy hệ thống có khả năng kết xuất các cảnh thử nghiệm ở tốc độ tương tác, đồng thời hội tụ dần về hình ảnh ít nhiễu khi camera giữ nguyên. Đồ án cũng chỉ ra các hướng tối ưu và mở rộng trong tương lai như cấu trúc tăng tốc BVH và các kỹ thuật giảm nhiễu nâng cao.

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

Nguyễn Xuân Khôi

Mục lục

1	Giới thiệu đề tài	1
1.1	Đặt vấn đề	1
1.2	Mục tiêu của đề tài	1
1.3	Phạm vi và đối tượng nghiên cứu	2
1.4	Cấu trúc báo cáo	2
2	Nền tảng lý thuyết	3
2.1	Đại lượng trắc quang và phương trình kết xuất	3
2.1.1	Góc khối và bức xạ	3
2.1.2	Hàm phản xạ lưỡng hướng (BRDF)	4
2.1.3	Phương trình kết xuất	4
2.2	Ước lượng Monte Carlo	4
2.2.1	Bộ ước lượng cơ bản	4
2.2.2	Lấy mẫu theo tầm quan trọng	5
2.2.3	Công thức tích phân đường và hệ số suy giảm	5
2.2.4	Russian Roulette	6
2.3	Lấy mẫu điểm ảnh và khử răng cưa	6
2.3.1	Lý thuyết lấy mẫu và hiện tượng răng cưa	6
2.3.2	Khử răng cưa bằng lấy mẫu ngẫu nhiên	7
2.3.3	Hiệu chỉnh gamma	8
2.4	Mô hình vật liệu	8
2.4.1	Vật liệu khuếch tán (Lambertian)	8
2.4.2	Vật liệu kim loại (Metal)	9
2.4.3	Vật liệu điện môi (Dielectric)	9
2.5	Mô hình camera thấu kính mỏng	9
2.5.1	Phương trình đo của camera	9
2.5.2	Từ lỗ kim tới thấu kính mỏng	10
2.5.3	Sinh tia với khẩu độ hữu hạn	10
2.5.4	Hệ trục và nhòe chuyển động	11
3	Triển khai hệ thống	12
3.1	Tổng quan kiến trúc hệ thống	12
3.2	Công nghệ và môi trường phát triển	12
3.3	Cầu nối CUDA–OpenGL và vòng lặp kết xuất	13

3.4	Hiện thực nhân Path Tracing trên CUDA	13
3.5	Hệ thống vật liệu	14
3.6	Mô hình camera	14
3.7	Quản lý cảnh và xây dựng thể giới trên GPU	14
3.8	Nạp cảnh từ định dạng Mitsuba 3	15
3.9	Tích lũy theo thời gian và hiệu chỉnh gamma	15
3.10	Giao diện điều khiển tương tác	15
4	Đánh giá	16
4.1	Môi trường thử nghiệm	16
4.2	Chất lượng kết xuất tổng thể	16
4.3	Tái tạo các loại vật liệu	17
4.4	Độ sâu trường ảnh	18
4.5	Nhòe chuyển động	19
4.6	Làm tối rìa ảnh (Vignetting)	19
4.7	Nhận xét chung	20
5	Kết luận và hướng phát triển	21
5.1	Kết luận	21
5.2	Hướng phát triển	21
5.3	Mã nguồn	22
	Tài liệu tham khảo	24

Danh sách hình vẽ

2.1	Hình học tại một điểm bề mặt: bức xạ tới theo hướng ω_i trong góc khối vi phân $d\omega_i$, bức xạ phản xạ theo hướng ω_o , với pháp tuyến $\mathbf{n}$ và góc tới θ_i	4
2.2	Một đường đi ngẫu nhiên trong Path Tracing: tia xuất phát từ camera, tán xạ qua các điểm va chạm $\mathbf{p}_1, \mathbf{p}_2, \dots$ và kết thúc tại nguồn sáng; hệ số suy giảm β được nhân dồn dọc theo đường.	6
2.3	Phổ tần số sau khi lấy mẫu gồm phổ gốc và các bản sao lặp lại. (a) Khi thỏa điều kiện Nyquist (2.8), các bản sao tách rời nên khôi phục được tín hiệu. (b) Khi lấy mẫu thiếu (hoặc tín hiệu không giới hạn dải tần), các bản sao chồng lấn, gây hiện tượng răng cưa.	7
2.4	Mô hình thấu kính mỏng. Điểm nằm trên mặt phẳng lấy nét (xanh) hội tụ thành một điểm sắc nét trên cảm biến; điểm ngoài mặt phẳng lấy nét (đỏ) trải ra thành vòng tròn nhòe, tạo hiệu ứng độ sâu trường ảnh.	10
4.1	Ảnh kết xuất trên cảnh mật độ cao sau 1846 khung hình tích lũy.	17
4.2	Ba loại vật liệu Lambertian, điện môi và kim loại trên cùng một cảnh.	18
4.3	Hiệu ứng độ sâu trường ảnh ở hai cấu hình khẩu độ và khoảng lấy nét.	18
4.4	Nhòe chuyển động trên các khối cầu được gán vận tốc.	19
4.5	Hiệu ứng làm tối rìa ảnh (vignetting): độ sáng giảm dần từ tâm ra các góc khung hình.	20

Chương 1

Giới thiệu đề tài

1.1 Đặt vấn đề

Kết xuất đồ họa là quá trình tạo ra hình ảnh hai chiều từ mô tả của một cảnh ba chiều. Trong số các kỹ thuật kết xuất, *Path Tracing* là phương pháp dựa trên mô phỏng vật lý của ánh sáng, cho phép tạo ra hình ảnh có độ chân thực cao với các hiệu ứng như phản xạ, khúc xạ, đổ bóng mềm và chiếu sáng toàn cục. Tuy nhiên, chi phí tính toán của Path Tracing rất lớn do phải mô phỏng hàng triệu tia sáng cho mỗi khung hình.

Chính vì chi phí đó, trong nhiều năm Path Tracing chỉ được dùng để kết xuất ngoại tuyến (offline): mỗi khung hình có thể mất từ vài phút tới hàng giờ trên CPU, phù hợp với điện ảnh và thiết kế nhưng không thể tương tác. Trong khi đó, nhu cầu về hình ảnh chân thực ở tốc độ thời gian thực ngày càng lớn, từ trò chơi điện tử, thực tại ảo cho tới các công cụ xem trước cảnh mà người dùng có thể xoay góc nhìn và chỉnh vật liệu ngay lập tức. Sự xuất hiện của các bộ xử lý đồ họa (GPU) với hàng nghìn nhân tính toán song song đã mở ra khả năng thu hẹp khoảng cách này: bản chất của Path Tracing là mỗi điểm ảnh được tính gần như độc lập, rất phù hợp với mô hình tính toán song song dữ liệu của GPU. Đề tài này khai thác đặc điểm đó để đưa Path Tracing tới tốc độ tương tác trên phần cứng phổ thông.

1.2 Mục tiêu của đề tài

Đề tài hướng tới việc nghiên cứu và xây dựng một bộ kết xuất Path Tracing chạy song song trên kiến trúc CUDA của NVIDIA nhằm đạt tốc độ tương tác (thời gian thực). Các mục tiêu cụ thể bao gồm:

- Hiện thực thuật toán Path Tracing trên GPU bằng CUDA.
- Xây dựng hệ thống vật liệu Lambertian, Metal, Dielectric và camera vật lý có độ sâu trường ảnh, nhòe chuyển động.
- Hiển thị thời gian thực qua cầu nối CUDA–OpenGL và tích lũy mẫu theo thời gian để khử nhiễu.
- Nạp cảnh theo định dạng Mitsuba 3 và cung cấp giao diện điều khiển tương tác.

1.3 Phạm vi và đối tượng nghiên cứu

Đối tượng nghiên cứu của đề tài là thuật toán Path Tracing và cách hiện thực nó chạy song song trên GPU bằng CUDA để đạt tốc độ tương tác. Trong khuôn khổ đó, phạm vi được giới hạn như sau:

- **Hình học cảnh:** chỉ hỗ trợ nguyên thủy khối cầu (tĩnh và chuyển động); các loại hình học khác như tam giác, lưới đa giác nằm ngoài phạm vi và được để lại cho hướng phát triển.
- **Vật liệu:** ba mô hình kinh điển gồm khuếch tán (Lambertian), kim loại (Metal) và điện môi (Dielectric); chưa xét các mô hình vi mặt phức tạp hay vật liệu phát sáng.
- **Chiếu sáng:** nguồn sáng dạng bầu trời gradient, dựa hoàn toàn vào chiếu sáng toàn cục qua nhiều lần dội tia thay vì nguồn sáng tường minh.
- **Định dạng nhập:** cảnh có thể dựng trực tiếp trong mã nguồn hoặc nạp từ tệp mô tả theo định dạng Mitsuba 3 (phần khối cầu và vật liệu).
- **Phần cứng:** hướng tới GPU NVIDIA phổ thông có hỗ trợ CUDA; toàn bộ thử nghiệm được thực hiện trên một máy tính xách tay cấu hình trung bình (chi tiết ở Chương 4).

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành năm chương. Chương 1 giới thiệu đề tài. Chương 2 trình bày nền tảng lý thuyết. Chương 3 mô tả quá trình triển khai hệ thống. Chương 4 trình bày kết quả và đánh giá. Chương 5 đưa ra kết luận và hướng phát triển.

Chương 2

Nền tảng lý thuyết

Chương này trình bày cơ sở lý thuyết của hệ thống. Trước hết ta xây dựng các đại lượng trắc quang và phương trình kết xuất mô tả sự cân bằng năng lượng ánh sáng. Tiếp theo là phương pháp Monte Carlo dùng để ước lượng tích phân trong phương trình đó, kèm các kỹ thuật giảm phương sai và lấy mẫu điểm ảnh. Phần sau trình bày mô hình hàm phản xạ (BRDF) cho ba loại vật liệu được hiện thực và mô hình camera thấu kính mỏng. Nội dung được tham khảo chủ yếu từ loạt sách *Ray Tracing in One Weekend* và sách *Physically Based Rendering*.

2.1 Đại lượng trắc quang và phương trình kết xuất

2.1.1 Góc khối và bức xạ

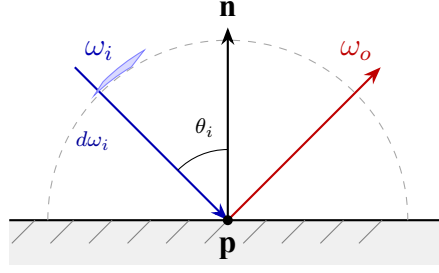
Để định lượng ánh sáng một cách độc lập với hình học, ta dùng khái niệm góc khối. Một vùng trên mặt cầu đơn vị chắn bởi một hướng nhìn có độ lớn góc khối $d\omega$ (đơn vị steradian). Trong hệ tọa độ cầu, vi phân góc khối quanh hướng (θ, ϕ) là

$$d\omega = \sin \theta \, d\theta \, d\phi. \quad (2.1)$$

Đại lượng trung tâm của kết xuất dựa trên vật lý là bức xạ L : công suất ánh sáng trên một đơn vị diện tích vuông góc, trên một đơn vị góc khối, ký hiệu $L(\mathbf{p}, \omega)$ tại điểm $\mathbf{p}$ theo hướng ω . Bức xạ là đại lượng được bảo toàn dọc theo tia trong môi trường chân không và là giá trị mà mỗi tia trong path tracer mang theo. Từ bức xạ tới, độ rọi E trên một bề mặt có pháp tuyến $\mathbf{n}$ thu được bằng cách lấy tích phân theo bán cầu, có trọng số cosine:

$$E(\mathbf{p}) = \int_{\Omega} L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) \, d\omega_i. \quad (2.2)$$

Thừa số cosine $(\omega_i \cdot \mathbf{n}) = \cos \theta_i$ phản ánh việc một chùm sáng tới xiên trải năng lượng trên diện tích lớn hơn nên đóng góp ít hơn.



Hình 2.1: Hình học tại một điểm bề mặt: bức xạ tới theo hướng ω_i trong góc khối vi phân $d\omega_i$, bức xạ phản xạ theo hướng ω_o , với pháp tuyến $\mathbf{n}$ và góc tới θ_i .

2.1.2 Hàm phản xạ lưỡng hướng (BRDF)

Cách một bề mặt phản xạ ánh sáng được mô tả bởi hàm phân bố phản xạ lưỡng hướng (BRDF), định nghĩa là tỉ số giữa bức xạ phản xạ theo hướng ω_o và độ rọi vi phân tới từ hướng ω_i :

$$f_r(\mathbf{p}, \omega_i, \omega_o) = \frac{dL_o(\mathbf{p}, \omega_o)}{L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i}. \quad (2.3)$$

Một BRDF hợp lệ về mặt vật lý phải thỏa mãn hai tính chất: tính đối xứng Helmholtz $f_r(\omega_i, \omega_o) = f_r(\omega_o, \omega_i)$, và bảo toàn năng lượng – tổng năng lượng phản xạ không vượt quá năng lượng tới.

2.1.3 Phương trình kết xuất

Kết hợp BRDF với độ rọi tới, bức xạ rời khỏi điểm $\mathbf{p}$ theo hướng ω_o được cho bởi phương trình kết xuất do Kajiya đề xuất:

$$L_o(\mathbf{p}, \omega_o) = L_e(\mathbf{p}, \omega_o) + \int_{\Omega} f_r(\mathbf{p}, \omega_i, \omega_o) L_i(\mathbf{p}, \omega_i) (\omega_i \cdot \mathbf{n}) d\omega_i, \quad (2.4)$$

trong đó L_e là bức xạ tự phát (vật liệu phát sáng), số hạng tích phân là bức xạ phản xạ. Đây là phương trình Fredholm loại hai: ẩn L xuất hiện ở cả hai vế vì bức xạ tới $L_i(\mathbf{p}, \omega_i)$ tại $\mathbf{p}$ chính là bức xạ đi ra $L_o(\mathbf{p}', -\omega_i)$ của điểm $\mathbf{p}'$ mà tia $(\mathbf{p}, \omega_i)$ chạm tới. Tính đệ quy này khiến lời giải giải tích là bất khả thi với cảnh tổng quát, do đó cần phương pháp số.

2.2 Ước lượng Monte Carlo

2.2.1 Bộ ước lượng cơ bản

Phương pháp Monte Carlo xấp xỉ một tích phân xác định bằng kỳ vọng thống kê. Với tích phân $I = \int g(x) dx$, nếu lấy N mẫu độc lập x_k theo mật độ xác suất $p(x)$ (với $p(x) > 0$ ở những nơi

$g(x) \neq 0$), bộ ước lượng Monte Carlo là

$$\langle I \rangle = \frac{1}{N} \sum_{k=1}^N \frac{g(x_k)}{p(x_k)}. \quad (2.5)$$

Bộ ước lượng này không chệch: $\mathbb{E}[\langle I \rangle] = I$ với mọi N . Phương sai của nó giảm tỉ lệ $1/N$, nên sai số chuẩn (độ lệch chuẩn) giảm theo $O(1/\sqrt{N})$. Hệ quả thực tiễn: muốn giảm một nửa nhiều cần gấp bốn lần số mẫu – đây là nguồn gốc của hiện tượng nhiễu hạt đặc trưng và chi phí tính toán lớn của path tracing.

2.2.2 Lấy mẫu theo tầm quan trọng

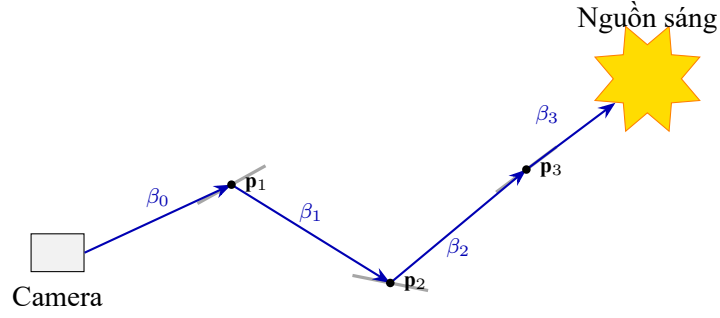
Phương sai phụ thuộc mạnh vào mức độ phù hợp giữa $p(x)$ và $g(x)$. Nếu chọn $p(x)$ tỉ lệ với $g(x)$ thì tỉ số g/p gần như hằng số và phương sai giảm mạnh; đây là ý tưởng lấy mẫu theo tầm quan trọng. Trong kết xuất, do tích phân (2.4) chứa thừa số cosine và BRDF, ta thường lấy mẫu hướng ω_i theo phân bố cosine hoặc theo hình dạng của BRDF thay vì đều trên bán cầu. Hệ thống hiện tại lấy mẫu khuếch tán theo phân bố cosine (Mục 2.4); việc lấy mẫu trực tiếp nguồn sáng là hướng mở rộng nêu ở Chương 5.

2.2.3 Công thức tích phân đường và hệ số suy giảm

Khai triển đệ quy phương trình (2.4) cho công thức tích phân đường: bức xạ tới camera là tổng đóng góp của mọi đường ánh sáng nối nguồn sáng tới mắt qua một số lần phản xạ. Path tracing ước lượng tổng này bằng cách, tại mỗi điểm va chạm, lấy một hướng tán xạ ngẫu nhiên và lần theo tia đó – tạo thành một đường đi ngẫu nhiên. Đóng góp của đường được tích lũy qua hệ số suy giảm β , cập nhật nhân dồn sau mỗi va chạm:

$$\beta_n = \beta_{n-1} \frac{f_r(\mathbf{p}_n, \omega_i, \omega_o) (\omega_i \cdot \mathbf{n})}{p(\omega_i)}, \quad \beta_0 = 1. \quad (2.6)$$

Khi tia thoát khỏi cảnh và gặp nguồn sáng nền, bức xạ thu được bằng tích của bức xạ nền với β . Với cách lấy mẫu khuếch tán theo cosine, thừa số cosine và mẫu số $p(\omega_i)$ triệt tiêu nhau, khiến việc cập nhật β rút gọn về phép nhân với albedo – đúng như hiện thực trong mã nguồn (Chương 3).



Hình 2.2: Một đường đi ngẫu nhiên trong Path Tracing: tia xuất phát từ camera, tán xạ qua các điểm và chạm $\mathbf{p}_1, \mathbf{p}_2, \dots$ và kết thúc tại nguồn sáng; hệ số suy giảm β được nhân dồn dọc theo đường.

2.2.4 Russian Roulette

Vì một đường có thể phản xạ vô hạn, cần điều kiện dừng. Cách đơn giản là giới hạn độ sâu tối đa, song điều này tạo ra sai lệch do bỏ qua các đường dài. Một giải pháp không chệch là Russian Roulette: tại mỗi bước, kết thúc đường với xác suất q và chia phần còn lại cho $1 - q$ để giữ kỳ vọng không đổi. Hệ thống áp dụng giới hạn độ sâu (tối đa 50 lần dội) kết hợp dừng sớm khi β suy giảm gần bằng 0 – một xấp xỉ thực dụng cho chế độ thời gian thực.

2.3 Lấy mẫu điểm ảnh và khử răng cưa

2.3.1 Lý thuyết lấy mẫu và hiện tượng răng cưa

Quá trình tạo ảnh số thực chất là lấy mẫu một hàm ảnh liên tục $f(x, y)$ – giá trị bức xạ tại mọi điểm trên mặt phẳng ảnh – tại một lưới điểm ảnh rời rạc, rồi tái dựng ảnh hiển thị từ các mẫu đó. Cơ sở lý thuyết của quá trình này là phân tích Fourier.

Mỗi hàm f có một phổ tần số $F = \mathcal{F}\{f\}$ thu được qua biến đổi Fourier. Việc lấy mẫu đều với bước T tương đương nhân f với một chuỗi xung Dirac khoảng cách T . Theo định lý tích chập, trong miền tần số phép nhân này trở thành tích chập của phổ F với một chuỗi xung khoảng cách $1/T$, tạo ra vô số bản sao của phổ gốc lặp lại theo chu kỳ $1/T$:

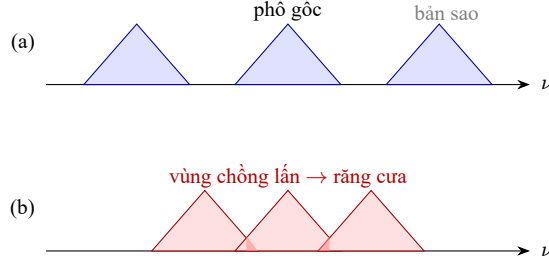
$$\mathcal{F}\left\{f \cdot \sum_n \delta(x - nT)\right\} = F * \frac{1}{T} \sum_k \delta\left(\nu - \frac{k}{T}\right). \quad (2.7)$$

Để khôi phục f , ta lọc lấy bản sao trung tâm bằng một bộ lọc thông thấp lý tưởng. Điều này khả thi nếu và chỉ nếu các bản sao không chồng lấn, tức hàm phải giới hạn dải tần với tần số lớn nhất $\nu_{\max}$, và tần số lấy mẫu thỏa

$$\frac{1}{T} > 2\nu_{\max}. \quad (2.8)$$

Bất đẳng thức (2.8) chính là định lý lấy mẫu Nyquist–Shannon; ngưỡng $2\nu_{\max}$ gọi là tần số Nyquist.

Trong kết xuất, điều kiện này gần như luôn bị vi phạm: biên hình học, bóng đổ sắc nét hay hoa văn texture tạo ra các bước nhảy có phổ trải tới vô hạn nên ảnh không giới hạn dải tần. Khi đó các bản sao phổ chồng lấn, năng lượng tần số cao “giả dạng” thành tần số thấp sau khi tái dựng, sinh ra các sai lệch nhìn thấy được gọi là răng cưa: biên xiên bị bậc thang, hoa văn nhỏ biến thành vân moiré.



Hình 2.3: Phổ tần số sau khi lấy mẫu gồm phổ gốc và các bản sao lặp lại. (a) Khi thỏa điều kiện Nyquist (2.8), các bản sao tách rời nên khôi phục được tín hiệu. (b) Khi lấy mẫu thiếu (hoặc tín hiệu không giới hạn dải tần), các bản sao chồng lấn, gây hiện tượng răng cưa.

2.3.2 Khử răng cưa bằng lấy mẫu ngẫu nhiên

Có hai hướng chống răng cưa. Hướng thứ nhất là tiền lọc – làm mượt hàm ảnh để giới hạn dải tần trước khi lấy mẫu – nhưng nhìn chung không khả thi vì hàm ảnh không có dạng giải tích. Hướng thứ hai, thực dụng hơn, là tăng tần số lấy mẫu: bắn nhiều tia trên mỗi điểm ảnh (siêu lấy mẫu) để đẩy tần số Nyquist lên cao, thu hẹp vùng bị răng cưa.

Điều quan trọng là path tracer dùng lấy mẫu ngẫu nhiên thay vì lưới đều. Với mẫu đều, năng lượng tần số cao chồng lấn tạo ra các vân răng cưa có cấu trúc; còn khi điểm lấy mẫu được làm nhiều ngẫu nhiên, năng lượng đó bị phân tán thành nhiễu không cấu trúc. Mắt người ít nhạy với nhiễu hơn nhiều so với các vân răng cưa có quy luật, nên đây là sự đánh đổi có lợi.

Về mặt hình thức, giá trị một điểm ảnh là tích phân của hàm ảnh nhân với một hàm lọc tái dựng r trên miền điểm ảnh, $I_{ij} = \int r(x, y) f(x, y) dx dy$. Path tracer ước lượng tích phân này bằng Monte Carlo (2.5) với các mẫu được làm nhiều ngẫu nhiên trong ô điểm ảnh. Mỗi khung dịch điểm bắn tia một lượng $(\xi_u, \xi_v) \in [0, 1)^2$,

$$s = \frac{i + \xi_u}{W}, \quad t = 1 - \frac{j + \xi_v}{H}, \quad (2.9)$$

với (i, j) là chỉ số điểm ảnh và $W \times H$ là độ phân giải (hệ thống dùng hàm lọc hộp, tức trọng số đều trong ô). Trung bình của nhiều mẫu vừa khử răng cưa biên, vừa hội tụ về giá trị tích phân đúng.

2.3.3 Hiệu chỉnh gamma

Toàn bộ phép tính ánh sáng trong path tracer diễn ra trong không gian màu tuyến tính: giá trị màu tỉ lệ thuận với bức xạ vật lý, nên cộng, nhân albedo hay lấy trung bình nhiều mẫu đều cho kết quả đúng về mặt năng lượng. Tuy nhiên giá trị tuyến tính này không thể gửi thẳng ra màn hình, vì hai mắt xích cuối của chuỗi hiển thị – cảm nhận của mắt người và đáp ứng của thiết bị hiển thị – đều phi tuyến.

Thứ nhất, mắt người nhạy với độ sáng theo quy luật gần lôgarit: ở vùng tối, một thay đổi bức xạ nhỏ đã gây khác biệt cảm nhận rõ, còn ở vùng sáng phải thay đổi lớn hơn nhiều mới nhận ra. Nếu lượng hóa màu tuyến tính thành 8 bit đều nhau, phần lớn 256 mức sẽ dồn vào vùng sáng mà mắt ít phân biệt, trong khi vùng tối – nơi mắt tinh nhất – lại quá ít mức, sinh hiện tượng phân dải (banding). Thứ hai, tín hiệu gửi tới thiết bị hiển thị được nó diễn giải theo một hàm lũy thừa: cường độ phát ra xấp xỉ $I \propto V^\gamma$ với giá trị điều khiển V và $\gamma \approx 2,2$ (quy ước từ thời màn hình CRT và được chuẩn sRGB kế thừa). Gửi thẳng giá trị tuyến tính ra màn hình sẽ khiến ảnh bị tối đi đúng theo hàm lũy thừa đó.

Để bù cả hai yếu tố, trước khi ghi ra khung hình ta mã hóa gamma màu tuyến tính bằng hàm mũ nghịch đảo:

$$C' = C^{1/\gamma}. \quad (2.10)$$

Phép này vừa “kéo giãn” vùng tối để phân bổ nhiều mức lượng hóa hơn cho nơi mắt nhạy, vừa triệt tiêu hàm lũy thừa của thiết bị hiển thị để ảnh hiện ra đúng độ sáng như tính toán. Hệ thống chọn $\gamma = 2$, tức lấy căn bậc hai từng kênh màu – một xấp xỉ rẻ và đủ tốt của chuẩn 2,2.

2.4 Mô hình vật liệu

Mỗi vật liệu định nghĩa một hàm tán xạ: cho tia tới, trả về tia tán xạ và hệ số suy giảm. Hệ thống hiện thực ba mô hình kinh điển của loạt *Ray Tracing in One Weekend*.

2.4.1 Vật liệu khuếch tán (Lambertian)

Bề mặt khuếch tán lý tưởng có BRDF hằng số:

$$f_r = \frac{\rho}{\pi}, \quad (2.11)$$

với ρ là albedo (hệ số phản xạ) và hệ số π bảo đảm bảo toàn năng lượng khi lấy tích phân theo bán cầu. Để lấy mẫu hiệu quả, ta sinh hướng tán xạ theo phân bố cosine bằng một mẹo hình học: cộng pháp tuyến với một điểm ngẫu nhiên trên mặt cầu đơn vị,

$$\omega_i = \mathbf{n} + \mathbf{r}_{\text{unit}}, \quad \|\mathbf{r}_{\text{unit}}\| = 1. \quad (2.12)$$

Khi đó mật độ lấy mẫu $p(\omega_i) = \cos \theta_i / \pi$ khớp với thừa số cosine và BRDF, làm phép cập nhật throughput (2.6) rút gọn còn phép nhân với albedo ρ .

2.4.2 Vật liệu kim loại (Metal)

Kim loại phản xạ gương: tia phản xạ quanh pháp tuyến theo

$$\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n}) \mathbf{n}, \quad (2.13)$$

với $\mathbf{d}$ là hướng tia tới đã chuẩn hóa. Để mô phỏng bề mặt nhám, hướng phản xạ được nhiễu loạn bằng một vector ngẫu nhiên trong quả cầu bán kính fuzz: $\mathbf{r}' = \mathbf{r} + \text{fuzz} \cdot \mathbf{r}_{\text{rand}}$. Giá trị fuzz càng lớn, phản chiếu càng mờ; tia tán xạ chui xuống dưới bề mặt sẽ bị loại bỏ.

2.4.3 Vật liệu điện môi (Dielectric)

Vật liệu trong suốt (thủy tinh, nước) vừa phản xạ vừa khúc xạ. Phần khúc xạ tuân theo định luật Snell:

$$\eta \sin \theta_i = \eta' \sin \theta_t, \quad (2.14)$$

với η, η' là chiết suất hai môi trường, θ_i, θ_t là góc tới và góc khúc xạ. Khi $\frac{\eta}{\eta'} \sin \theta_i > 1$, phương trình (2.14) vô nghiệm với $\sin \theta_t$: xảy ra phản xạ toàn phần, tia buộc phải phản xạ theo (2.13).

Tỉ lệ năng lượng phản xạ so với khúc xạ được cho bởi phương trình Fresnel, vốn phức tạp vì phụ thuộc phân cực. Trong đồ họa, ta dùng xấp xỉ Schlick chính xác cao và rẻ về tính toán:

$$R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5, \quad R_0 = \left(\frac{\eta - \eta'}{\eta + \eta'} \right)^2, \quad (2.15)$$

trong đó R_0 là hệ số phản xạ ở góc tới vuông góc. Thay vì tính cả hai tia phản xạ và khúc xạ rồi pha trộn, path tracer dùng cách lấy mẫu ngẫu nhiên: sinh một số $\xi \in [0, 1)$ và chọn phản xạ nếu $\xi < R(\theta)$, ngược lại thì khúc xạ. Theo nghĩa thống kê qua nhiều mẫu, tỉ lệ này tái tạo đúng hệ số Fresnel mà mỗi tia chỉ phải lần theo một nhánh.

2.5 Mô hình camera thấu kính mỏng

2.5.1 Phương trình đo của camera

Một cách hình thức, giá trị mà một điểm ảnh ghi nhận không phải là bức xạ của một tia đơn lẻ, mà là một phép đo trên toàn bộ ánh sáng đi vào hệ quang học. Theo PBRT, đáp ứng I_j của điểm ảnh thứ j được cho bởi phương trình đo:

$$I_j = \int_{A_{\text{film}}} \int_{\Omega} W_e^{(j)}(\mathbf{p}, \omega) L_i(\mathbf{p}, \omega) |\cos \theta| d\omega dA(\mathbf{p}), \quad (2.16)$$

trong đó tích phân lấy trên diện tích phim A_{film} và bán cầu hướng Ω ; L_i là bức xạ tới, còn $W_e^{(j)}$ là hàm tầm quan trọng – đặc trưng độ nhạy của cảm biến với tia $(\mathbf{p}, \omega)$, đóng vai trò đối ngẫu với bức xạ phát ra của một nguồn sáng. Nhiệm vụ của mô hình camera là sinh các tia sao cho việc lấy mẫu Monte Carlo tích phân (2.16) là hợp lệ.

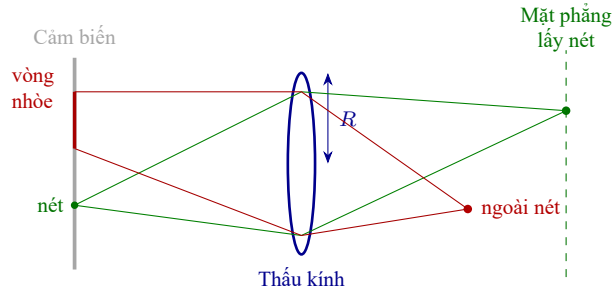
2.5.2 Từ lỗ kim tới thấu kính mỏng

Với camera lỗ kim lý tưởng, khẩu độ thu về một điểm: hàm tầm quan trọng W_e suy biến thành một xung Dirac theo hướng, nên mỗi điểm trên phim chỉ ứng với một tia duy nhất và mọi vật thể đều sắc nét bất kể độ sâu. Đây là lý do ảnh pinhole không có hiệu ứng nhòe của ống kính thật.

Để tái hiện độ sâu trường ảnh, ta cần một khẩu độ hữu hạn. Mô hình thấu kính mỏng xấp xỉ hệ quang học bằng một thấu kính lý tưởng độ dày bằng 0, tuân theo phương trình thấu kính Gauss:

$$\frac{1}{z'} - \frac{1}{z} = \frac{1}{f}, \quad (2.17)$$

với z là khoảng cách từ vật tới thấu kính, z' là khoảng cách ảnh, và f là tiêu cự. Phương trình (2.17) chỉ ra rằng các vật ở những khoảng cách z khác nhau hội tụ tại những mặt phẳng ảnh z' khác nhau. Do cảm biến đặt cố định, chỉ những vật nằm tại mặt phẳng lấy nét (khoảng cách focus distance) mới hội tụ thành điểm sắc nét; vật ở khoảng cách khác trải thành một đĩa gọi là vòng tròn nhòe, gây hiệu ứng mờ nhòe. Bán kính vòng nhòe – và do đó độ mạnh của hiệu ứng – tỉ lệ với bán kính khẩu độ.



Hình 2.4: Mô hình thấu kính mỏng. Điểm nằm trên mặt phẳng lấy nét (xanh) hội tụ thành một điểm sắc nét trên cảm biến; điểm ngoài mặt phẳng lấy nét (đỏ) trải ra thành vòng tròn nhòe, tạo hiệu ứng độ sâu trường ảnh.

2.5.3 Sinh tia với khẩu độ hữu hạn

Tích phân (2.16) trên diện tích thấu kính được ước lượng bằng Monte Carlo: thay vì một tia, ta lấy mẫu một điểm trên đĩa thấu kính bán kính $R = \text{aperture}/2$. Quy trình sinh tia gồm ba bước:

1. Dựng tia chính đi từ tâm thấu kính qua điểm ảnh (s, t) .
2. Xác định điểm lấy nét là giao của tia chính với mặt phẳng lấy nét (cách camera một khoảng focus distance) – điểm này luôn sắc nét.

3. Lấy ngẫu nhiên một điểm trên đĩa thấu kính làm gốc tia mới, hướng tia mới đi qua đúng điểm lấy nét ở bước 2.

Vì mọi tia lấy mẫu đều đi qua cùng điểm lấy nét, vật tại mặt phẳng lấy nét vẫn sắc nét; còn vật ngoài mặt phẳng đó nhận các tia từ những vị trí thấu kính khác nhau nên bị nhòe, đúng như mô tả của vòng tròn nhòe. Khi khẩu độ bằng 0, mô hình thoái biến về camera lỗ kim.

2.5.4 Hệ trục và nhòe chuyển động

Trường nhìn dọc θ xác định chiều cao khung nhìn qua $h = \tan(\theta/2)$. Hệ trục camera trục chuẩn $(\mathbf{u}, \mathbf{v}, \mathbf{w})$ được dựng từ vị trí đặt máy (lookfrom), điểm nhìn (lookat) và vector hướng lên (vup):

$$\mathbf{w} = \frac{\text{lookfrom} - \text{lookat}}{\|\cdot\|}, \quad \mathbf{u} = \frac{\text{vup} \times \mathbf{w}}{\|\cdot\|}, \quad \mathbf{v} = \mathbf{w} \times \mathbf{u}. \quad (2.18)$$

Để mô phỏng nhòe chuyển động, phương trình đo (2.16) được mở rộng thêm một tích phân theo thời gian phơi sáng. Hệ thống ước lượng tích phân này bằng cách gán cho mỗi tia một mốc thời gian ngẫu nhiên trong khoảng $[t_0, t_1]$; vật thể chuyển động được nội suy vị trí tuyến tính theo thời gian của tia, nên ảnh tích lũy thể hiện vật mờ tự nhiên.

Chương 3

Triển khai hệ thống

Chương này mô tả cách hiện thực các khái niệm ở Chương 2 thành một hệ thống chạy được. Nội dung lần lượt trình bày kiến trúc tổng thể và công nghệ sử dụng; cầu nối giữa CUDA và OpenGL cùng vòng lặp kết xuất thời gian thực; nhân Path Tracing và cấu hình thực thi trên GPU; hệ thống vật liệu và mô hình camera; cách quản lý cảnh và dựng thể giới trên GPU; bộ nạp cảnh theo định dạng Mitsuba 3; cơ chế tích lũy theo thời gian kèm hiệu chỉnh gamma; và cuối cùng là giao diện điều khiển tương tác. Phần trình bày tập trung mô tả cách tổ chức, cấu trúc dữ liệu và luồng xử lý ở mức tổng quan, thay vì đi sâu vào mã nguồn chi tiết.

3.1 Tổng quan kiến trúc hệ thống

Hệ thống được chia thành ba tầng phối hợp với nhau trong mỗi khung hình:

- **Tầng host (CPU, C++):** quản lý vòng lặp ứng dụng, cấp phát bộ nhớ GPU, nạp cảnh, xử lý nhập liệu và điều khiển camera.
- **Tầng device (GPU, CUDA):** thực thi nhân Path Tracing, tính màu cho từng điểm ảnh và ghi kết quả trực tiếp vào bộ nhớ video (VRAM).
- **Tầng hiển thị (OpenGL + ImGui):** đưa ảnh từ VRAM lên màn hình và vẽ bảng điều khiển tương tác.

Điểm mấu chốt về hiệu năng là dữ liệu ảnh không phải đi vòng qua CPU: CUDA ghi thẳng vào bộ đệm mà OpenGL dùng để hiển thị, nhờ cơ chế CUDA–OpenGL Interop trình bày ở Mục 3.3.

3.2 Công nghệ và môi trường phát triển

Hệ thống được xây dựng bằng C++ và CUDA, cấu hình biên dịch qua CMake.

- Ngôn ngữ: C++ (tầng host) và CUDA C (nhân thiết bị).
- CUDA Toolkit phiên bản ≥ 11.0 , biên dịch bằng nvcc.
- CMake phiên bản ≥ 3.18 .
- Hiển thị: OpenGL kết hợp GLFW (quản lý cửa sổ, nhập liệu).
- Giao diện người dùng: thư viện ImGui.
- Sinh số ngẫu nhiên trên GPU: thư viện cuRAND.
- Phân tích cảnh Mitsuba: thư viện tinyxml2.

3.3 Cầu nối CUDA–OpenGL và vòng lặp kết xuất

Để hiển thị thời gian thực mà không tốn chi phí sao chép qua lại giữa CPU và GPU, hệ thống dùng cơ chế interop. Một Pixel Buffer Object (PBO) được tạo bởi OpenGL nằm trên VRAM, sau đó đăng ký với CUDA bằng `cudaGraphicsGLRegisterBuffer`. Từ đó, CUDA có thể ghi thẳng kết quả vào PBO, còn OpenGL dán PBO lên một texture để hiển thị.

Trong mỗi vòng lặp khung hình, hệ thống thực hiện bốn bước:

1. Cập nhật giao diện ImGui và camera theo thao tác chuột/bàn phím; nếu có thay đổi thì đồng bộ camera xuống GPU và đặt lại bộ đếm tích lũy.
2. Khóa (map) PBO, lấy con trỏ VRAM rồi gọi nhân kết xuất để tô màu toàn bộ điểm ảnh; sau đó nhả khóa (unmap).
3. OpenGL sao chép PBO vào texture (bằng DMA phần cứng) và vẽ một hình chữ nhật phủ kín cửa sổ.
4. Vẽ đè giao diện ImGui và hoán đổi bộ đệm (double buffering).

3.4 Hiện thực nhân Path Tracing trên CUDA

Nhân kết xuất chính ánh xạ mỗi luồng GPU tới đúng một điểm ảnh thông qua chỉ số toàn cục tính từ chỉ số khối và chỉ số luồng. Mỗi luồng nạp trạng thái sinh số ngẫu nhiên riêng của điểm ảnh mình phụ trách, làm nhiều nhẹ vị trí bắn tia trong ô điểm ảnh để khử răng cưa, dựng tia tương ứng từ camera rồi gọi hàm dò đường để tính màu.

Hàm dò đường được hiện thực theo kiểu lặp thay cho đệ quy, vì đệ quy sâu tốn ngăn xếp và không phù hợp với GPU. Thay vì gọi lại chính nó sau mỗi va chạm, hàm duy trì một hệ số suy giảm β và nhân dồn nó với hệ số phản xạ của vật liệu tại mỗi lần dội, đúng theo công thức throughput ở Chương 2. Vòng lặp kết thúc trong ba trường hợp: tia thoát ra ngoài cảnh và nhận màu bầu trời (nhân với β để trả về đóng góp của đường), vật liệu hấp thụ hoàn toàn không tán xạ tiếp, hoặc số lần dội chạm ngưỡng tối đa (50) để bảo đảm luôn dừng. Khi tìm giao, cận dưới tham số tia được đặt là một giá trị dương nhỏ ($t_{\min} = 0,001$) nhằm tránh hiện tượng shadow acne – tia thứ cấp tự giao lại chính bề mặt vừa xuất phát do sai số dấu phẩy động.

Về cấu hình thực thi, nhân được khởi chạy trên một lưới hai chiều phủ toàn bộ khung hình, mỗi khối gồm $8 \times 8 = 64$ luồng tương ứng một mảng điểm ảnh liền kề. Cách chia khối này cân bằng giữa mức độ chiếm dụng của đa xử lý luồng và tính cục bộ khi truy cập bộ nhớ. Vì mỗi điểm ảnh cần một dãy số ngẫu nhiên độc lập cho quá trình lấy mẫu Monte Carlo, hệ thống dùng thư viện `cuRAND` và gieo hạt giống riêng cho từng điểm ảnh trong một nhân khối tạo chạy một lần trước vòng lặp; trạng thái này được lưu lại và tái sử dụng qua các khung. Một đặc thù cần lưu ý khi tối ưu là sự phân kỳ luồng: các tia trong cùng một nhóm luồng có thể chạm vật liệu khác loại hoặc dừng dò đường ở số lần dội khác nhau, khiến GPU phải thực thi tuần tự các nhánh và làm giảm hiệu suất so với trường hợp lý tưởng.

3.5 Hệ thống vật liệu

Vật liệu được biểu diễn bằng một struct `Material` gọn nhẹ gồm kiểu vật liệu, albedo, hệ số nhám fuzz và chiết suất `ir`. Cách dùng struct phẳng (thay vì lớp ảo) giúp truyền dữ liệu xuống GPU thuận lợi. Một hàm tán xạ chung phân nhánh theo kiểu vật liệu để xác định tia tán xạ:

- **Lambertian:** hướng tán xạ là pháp tuyến cộng một vector đơn vị ngẫu nhiên; nếu hướng thu được suy biến (gần bằng 0) thì thay bằng pháp tuyến.
- **Metal:** phản xạ gương quanh pháp tuyến, cộng thêm nhiễu trong quả cầu bán kính fuzz để mô phỏng bề mặt nhám; chỉ giữ lại tia còn hướng ra ngoài bề mặt.
- **Dielectric:** tính góc tới, xét điều kiện phản xạ toàn phần và so sánh xác suất Schlick (2.15) với một số ngẫu nhiên để chọn giữa phản xạ và khúc xạ theo định luật Snell; nhờ vậy mỗi tia chỉ đi theo một nhánh mà qua nhiều mẫu vẫn tái tạo đúng tỉ lệ Fresnel.

3.6 Mô hình camera

Lớp camera tiền tính các vector khung nhìn từ vị trí đặt máy, điểm nhìn, trường nhìn, tỉ lệ khung hình, khẩu độ và khoảng lấy nét. Với mỗi tọa độ ảnh, hàm sinh tia lấy ngẫu nhiên một điểm trên đĩa thấu kính làm gốc tia (tạo độ sâu trường ảnh) và hướng tia đi qua điểm lấy nét tương ứng, đồng thời gán cho tia một mốc thời gian ngẫu nhiên trong khoảng phơi sáng $[t_0, t_1]$ để phục vụ nhòe chuyển động. Cách sinh tia này hiện thực trực tiếp mô hình thấu kính mỏng và tích phân theo thời gian đã trình bày ở Chương 2.

3.7 Quản lý cảnh và xây dựng thế giới trên GPU

Trên phía host, mỗi cảnh được mô tả bằng một danh sách cấu trúc dữ liệu gọn nhẹ, mỗi phần tử ghi tâm, bán kính, vật liệu và thông tin chuyển động của một khối cầu. Danh sách này được sao chép xuống bộ nhớ thiết bị, sau đó một nhân khởi tạo chạy một lần sẽ dựng các đối tượng khối cầu (tĩnh hoặc chuyển động) ngay trên GPU và gom chúng vào một danh sách vật thể chung. Việc cấp phát và dựng đối tượng trực tiếp trên thiết bị cho phép sử dụng cơ chế đa hình (hàm ảo tìm giao) ngay trong nhân kết xuất mà không phải sao chép con trỏ qua lại giữa hai không gian bộ nhớ.

Khi dò tia, danh sách vật thể được duyệt tuyến tính: với mỗi tia, hệ thống lần lượt kiểm tra giao với từng khối cầu và luôn giữ lại giao điểm gần nhất bằng cách thu hẹp dần cận trên của tham số tia. Mỗi khối cầu được kiểm tra giao bằng cách giải phương trình bậc hai giữa tia và mặt cầu như đã nêu ở Chương 2, rồi điền vào bản ghi va chạm vị trí, pháp tuyến và vật liệu tại điểm giao. Cách duyệt tuyến tính có độ phức tạp tỉ lệ với số vật thể nên trở thành nút thắt hiệu năng khi cảnh đông đối tượng; đây chính là động lực cho việc tích hợp cấu trúc tăng tốc không gian (BVH) nêu ở Chương 5.

3.8 Nạp cảnh từ định dạng Mitsuba 3

Để tái sử dụng các cảnh chuẩn và tách dữ liệu cảnh khỏi mã nguồn, hệ thống hiện thực một bộ nạp phân tích tệp mô tả cảnh theo định dạng Mitsuba 3 bằng thư viện `tinyxml2`. Bộ nạp duyệt các thẻ con của cảnh và ánh xạ chúng sang mô hình nội bộ: thẻ cảm biến cung cấp trường nhìn để dựng camera; thẻ vật liệu được ánh xạ theo loại (khúc xạ sang Lambertian, điện môi sang Dielectric, dẫn điện sang Metal) và lưu lại theo định danh để tham chiếu; thẻ hình học với khối cầu cho biết bán kính, còn tâm được lấy từ cột tịnh tiến của ma trận biến đổi 4×4 , và vật liệu được liên kết qua định danh đã lưu. Phiên bản hiện tại tập trung hỗ trợ khối cầu; các loại hình học khác được bỏ qua kèm cảnh báo và là hướng mở rộng tự nhiên trong tương lai.

3.9 Tích lũy theo thời gian và hiệu chỉnh gamma

Do mỗi khung chỉ lấy một mẫu cho mỗi điểm ảnh, ảnh của một khung đơn lẻ rất nhiễu. Hệ thống duy trì một bộ đệm tích lũy lưu tổng màu qua các khung; màu hiển thị là trung bình cộng của tổng này chia cho số khung đã tích lũy, đúng theo bộ ước lượng Monte Carlo nên ảnh hội tụ dần và bớt nhiễu theo thời gian khi camera đứng yên. Mỗi khi camera hoặc một tham số kết xuất thay đổi, host đặt lại bộ đếm khung về 1 để loại bỏ những mẫu cũ không còn hợp lệ với góc nhìn mới. Trước khi ghi ra bộ đệm hiển thị, giá trị màu tuyến tính được hiệu chỉnh gamma (lấy căn bậc hai từng kênh) rồi quy về số nguyên 8-bit cho mỗi kênh RGBA, đúng như lý do đã phân tích ở Chương 2.

3.10 Giao diện điều khiển tương tác

Hệ thống cung cấp một bảng điều khiển dựng bằng thư viện `ImGui`, cho phép người dùng theo dõi và tinh chỉnh quá trình kết xuất theo thời gian thực. Bảng điều khiển hiển thị các chỉ số hiệu năng (số khung tích lũy, thời gian mỗi khung và số khung hình trên giây) và cho phép chỉnh trực tiếp các tham số camera như trường nhìn, khẩu độ, khoảng lấy nét cùng hệ số làm tối rìa ảnh. Song song, một bộ điều khiển camera xử lý thao tác chuột để xoay quanh tâm cảnh, tịnh tiến và phóng to/thu nhỏ. Bất kỳ thay đổi nào từ giao diện hay từ thao tác camera đều kích hoạt đồng bộ tham số camera xuống GPU và đặt lại quá trình tích lũy, bảo đảm ảnh luôn nhất quán với góc nhìn và cấu hình hiện tại.

Chương 4

Đánh giá

Chương này trình bày kết quả thực nghiệm của hệ thống đã hiện thực ở Chương 3. Phần đầu mô tả cấu hình phần cứng và phần mềm dùng để thử nghiệm. Phần sau phân tích các ảnh kết xuất theo từng nhóm hiệu ứng: chất lượng tổng thể, tái tạo vật liệu, độ sâu trường ảnh, nhòe chuyển động và làm tối rìa ảnh, kèm theo số liệu hiệu năng đo trực tiếp trong quá trình kết xuất.

4.1 Môi trường thử nghiệm

Toàn bộ thử nghiệm được thực hiện trên một máy tính xách tay với cấu hình nêu ở Bảng 4.1. Card đồ họa thuộc phân khúc phổ thông, dung lượng bộ nhớ video giới hạn ở 4 GB, nên kết quả hiệu năng phản ánh mức trung bình thấp của phần cứng người dùng phổ thông.

Bảng 4.1: Cấu hình môi trường thử nghiệm.

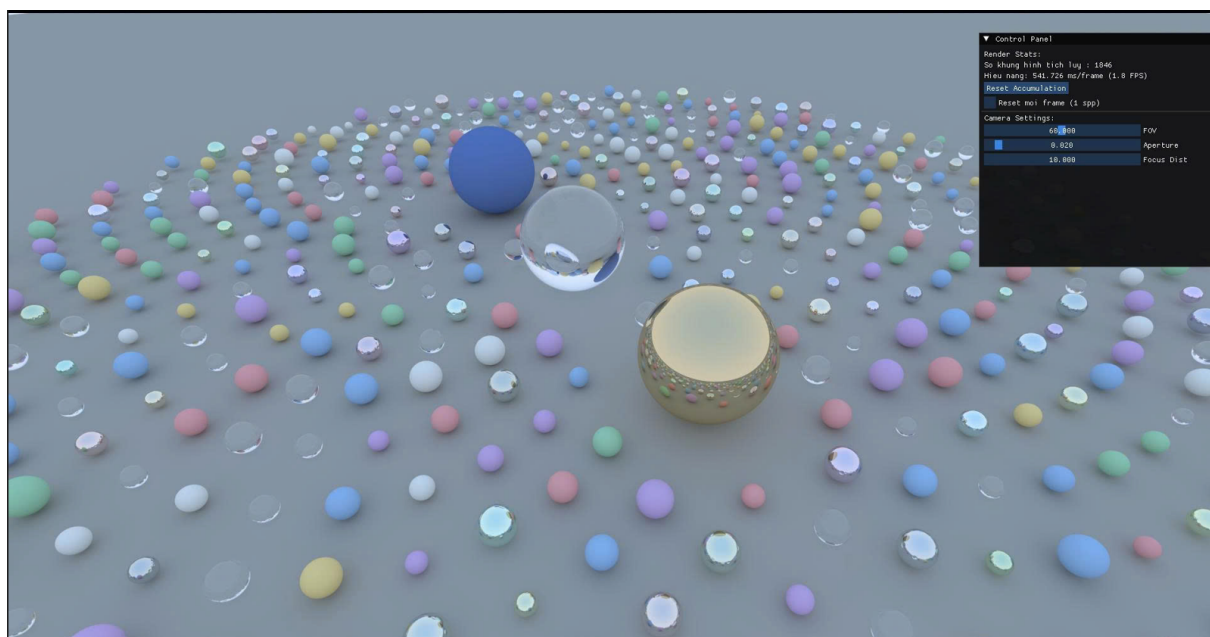
Thành phần	Thông số
CPU	AMD Ryzen 5 5600H, 6 nhân 12 luồng
GPU	NVIDIA GeForce GTX 1650, 4 GB GDDR
Bộ nhớ hệ thống	16 GB
Hệ điều hành	Windows 11, 64-bit
CUDA Toolkit	phiên bản 12.6
Trình biên dịch	MSVC kết hợp nvcc
Độ phân giải kết xuất	1920×1014 điểm ảnh

Hiệu năng được đo bằng hai chỉ số hiển thị trực tiếp trên bảng điều khiển: thời gian kết xuất trung bình của một khung hình tính bằng mili giây và số khung hình trên giây tương ứng. Mỗi khung hình lấy một mẫu cho mỗi điểm ảnh, sau đó cộng dồn vào bộ đệm tích lũy nên ảnh hội tụ dần theo thời gian. Số khung hình tích lũy được ghi nhận để thể hiện mức độ hội tụ của ảnh tại thời điểm chụp.

4.2 Chất lượng kết xuất tổng thể

Hình 4.1 là ảnh kết xuất trên cảnh có mật độ vật thể cao gồm vài trăm khối cầu với đủ ba loại vật liệu. Nguyên lý cốt lõi quyết định chất lượng ảnh ở đây là tích lũy Monte Carlo theo thời gian:

mỗi khung chỉ lấy một mẫu ngẫu nhiên cho mỗi điểm ảnh nên rất nhiều, nhưng khi lấy trung bình N mẫu độc lập qua các khung liên tiếp, phương sai của bộ ước lượng (2.5) giảm theo $1/N$ và biên độ nhiễu (độ lệch chuẩn) giảm theo $1/\sqrt{N}$. Chính quy luật $O(1/\sqrt{N})$ này giải thích vì sao ảnh hội tụ nhanh lúc đầu rồi chậm dần: sau 1846 khung hình tích lũy, nhiễu hạt gần như bị khử hoàn toàn và các hiệu ứng phản xạ, khúc xạ, đổ bóng mềm được tái tạo nhất quán. Muốn giảm tiếp một nửa nhiễu còn lại cần gấp bốn lần số khung, đúng như hệ quả của quy luật này. Ở cảnh đông đối tượng như vậy, thời gian mỗi khung tăng lên khoảng 542 ms tương ứng 1,8 khung hình trên giây, do mỗi tia phải duyệt tuyến tính qua toàn bộ danh sách vật thể để tìm giao điểm. Đây là hệ quả trực tiếp của việc chưa dùng cấu trúc tăng tốc không gian, và là động lực cho hướng tối ưu bằng BVH nêu ở Chương 5.

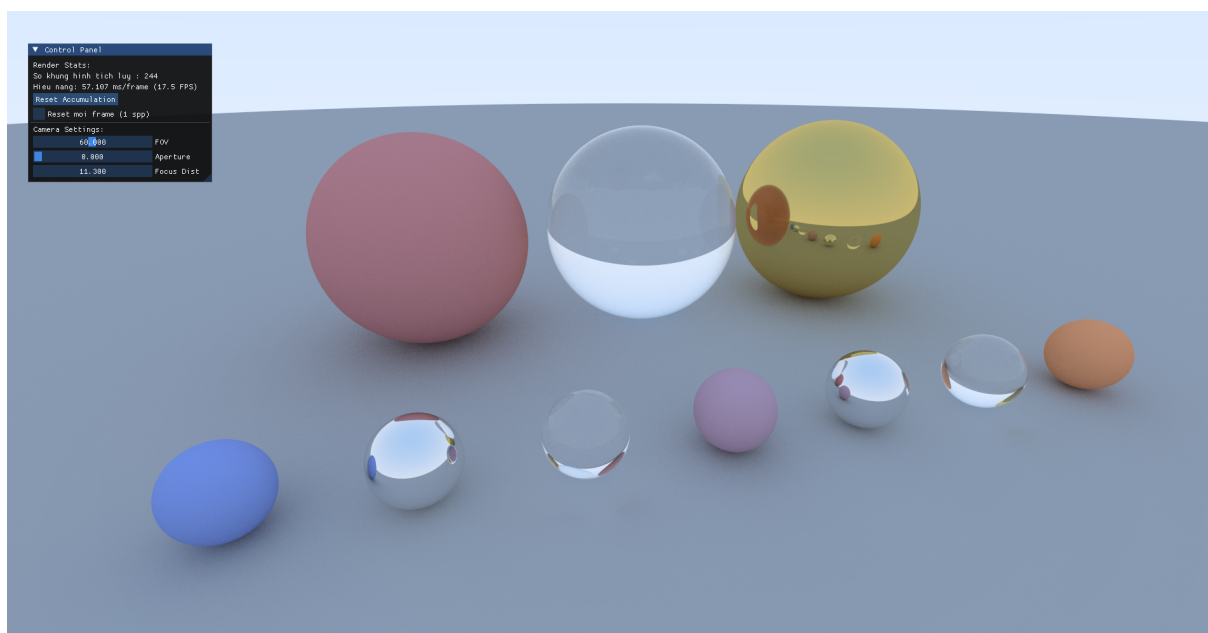


Hình 4.1: Ảnh kết xuất trên cảnh mật độ cao sau 1846 khung hình tích lũy.

4.3 Tái tạo các loại vật liệu

Hình 4.2 kiểm chứng riêng ba loại vật liệu đã hiện thực trên một cảnh thưa để quan sát rõ từng đặc tính, mỗi loại tương ứng một cơ chế tán xạ khác nhau. Khối cầu màu đỏ là vật liệu Lambertian: bề mặt khuếch tán lý tưởng có hàm phản xạ hằng số $f_r = \rho/\pi$, tán xạ ánh sáng đều theo mọi hướng nên bề mặt sáng đục và không phản chiếu môi trường. Khối cầu trong suốt là vật liệu điện môi: phần khúc xạ tuân theo định luật Snell (2.14) bẻ cong tia qua thân cầu làm đảo ngược ảnh nền, còn tỉ lệ năng lượng phản xạ ở rìa tăng vọt theo xấp xỉ Schlick $R(\theta) = R_0 + (1 - R_0)(1 - \cos \theta)^5$ (2.15) — đó là lý do viền cầu sáng và phản chiếu rõ hơn phần giữa. Khối cầu vàng kim là vật liệu kim loại: tia phản xạ gương quanh pháp tuyến theo $\mathbf{r} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n})\mathbf{n}$ (2.13) nên bề mặt ánh lên hình ảnh các vật thể xung quanh. Tại cấu hình khẩu độ bằng 0, toàn ảnh sắc nét và hệ thống đạt khoảng 57 ms mỗi khung tương ứng 17,5 khung hình

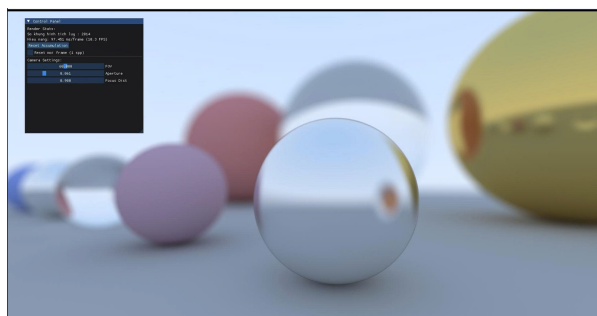
trên giây nhòr số lượng vật thể ít.



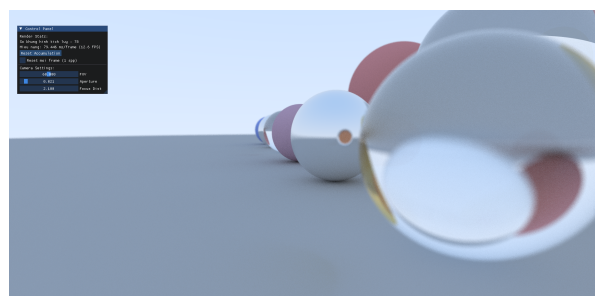
Hình 4.2: Ba loại vật liệu Lambertian, điện môi và kim loại trên cùng một cảnh.

4.4 Độ sâu trường ảnh

Hình 4.3 minh họa hai mức khẩu độ khác nhau. Nguyên lý cốt lõi của hiệu ứng này là khẩu độ hữu hạn của mô hình thấu kính mỏng: mỗi tia không xuất phát từ một điểm mà từ một vị trí lấy mẫu ngẫu nhiên trên đĩa thấu kính bán kính $R = \text{khẩu độ}/2$, và mọi tia đều buộc đi qua cùng một điểm trên mặt phẳng lấy nét. Nhờ vậy vật nằm đúng mặt phẳng lấy nét luôn hội tụ thành một điểm sắc nét; còn vật ở trước hoặc sau mặt phẳng đó nhận các tia đến từ những vị trí thấu kính khác nhau nên trải ra thành một đĩa gọi là vòng tròn nhòe. Bán kính vòng nhòe — và do đó mức độ mờ — tỉ lệ thuận với bán kính khẩu độ R : đó là lý do khẩu độ càng lớn thì vùng rõ nét càng hẹp và hậu cảnh càng mờ. Hiệu ứng phát sinh tự nhiên từ việc lấy mẫu trên đĩa thấu kính, không cần xử lý hậu kỳ; khi khẩu độ bằng 0 mô hình thoái biến về camera lỗ kim và toàn ảnh sắc nét.



(a) Khẩu độ 0,061, khoảng lấy nét 0,9.

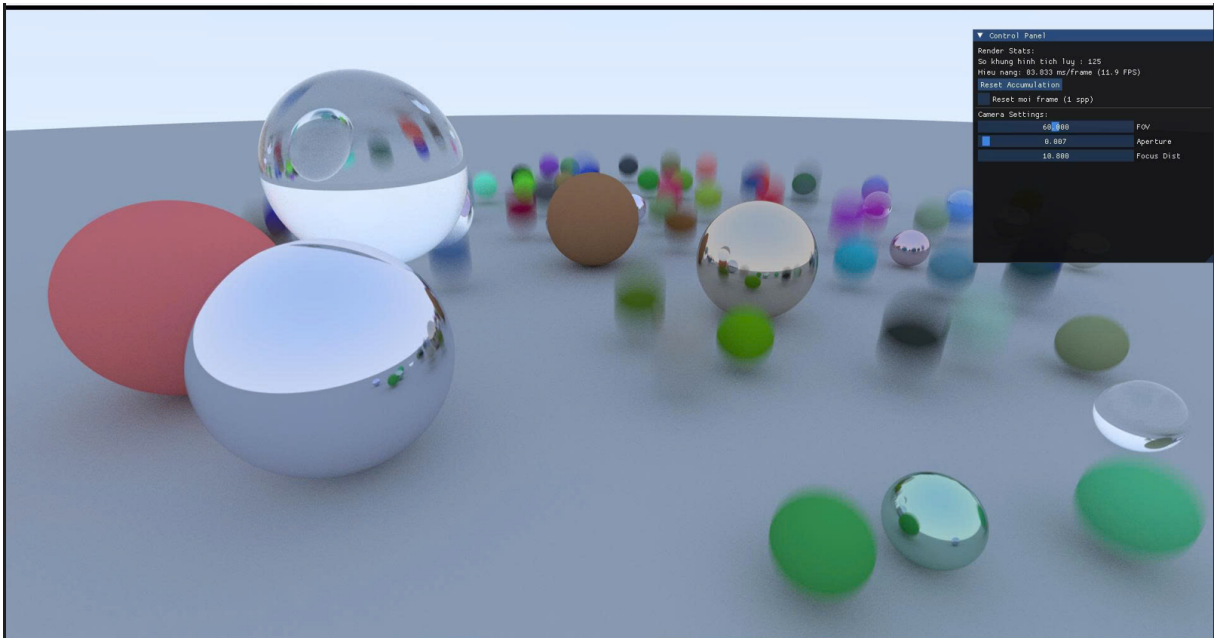


(b) Khẩu độ 0,021, khoảng lấy nét 2,1.

Hình 4.3: Hiệu ứng độ sâu trường ảnh ở hai cấu hình khẩu độ và khoảng lấy nét.

4.5 Nhòe chuyển động

Hình 4.4 kiểm chứng hiệu ứng nhòe chuyển động trên cảnh có một số khối cầu di chuyển. Nguyên lý cốt lõi là camera phơi sáng trong một khoảng thời gian hữu hạn: giá trị một điểm ảnh không phải bức xạ tại một thời điểm mà là tích phân của bức xạ theo suốt thời gian màn trập mở $[t_0, t_1]$. Hệ thống ước lượng tích phân thời gian này bằng Monte Carlo — mỗi tia mang một mốc thời gian ngẫu nhiên $t \in [t_0, t_1]$, và vị trí khối cầu chuyển động được nội suy tuyến tính $\mathbf{p}(t) = \mathbf{p}_0 + t \mathbf{v}$ theo đúng mốc đó. Vì trong một lần phơi sáng vật đã dịch qua cả quãng đường, năng lượng của nó trải đều dọc quỹ đạo thay vì đọng tại một chỗ, tạo thành vệt nhòe liên tục; màn trập mở càng lâu (khoảng $t_1 - t_0$ càng lớn) thì vệt nhòe càng dài. Các khối cầu tĩnh không đổi vị trí theo t nên vẫn sắc nét.



Hình 4.4: Nhòe chuyển động trên các khối cầu được gán vận tốc.

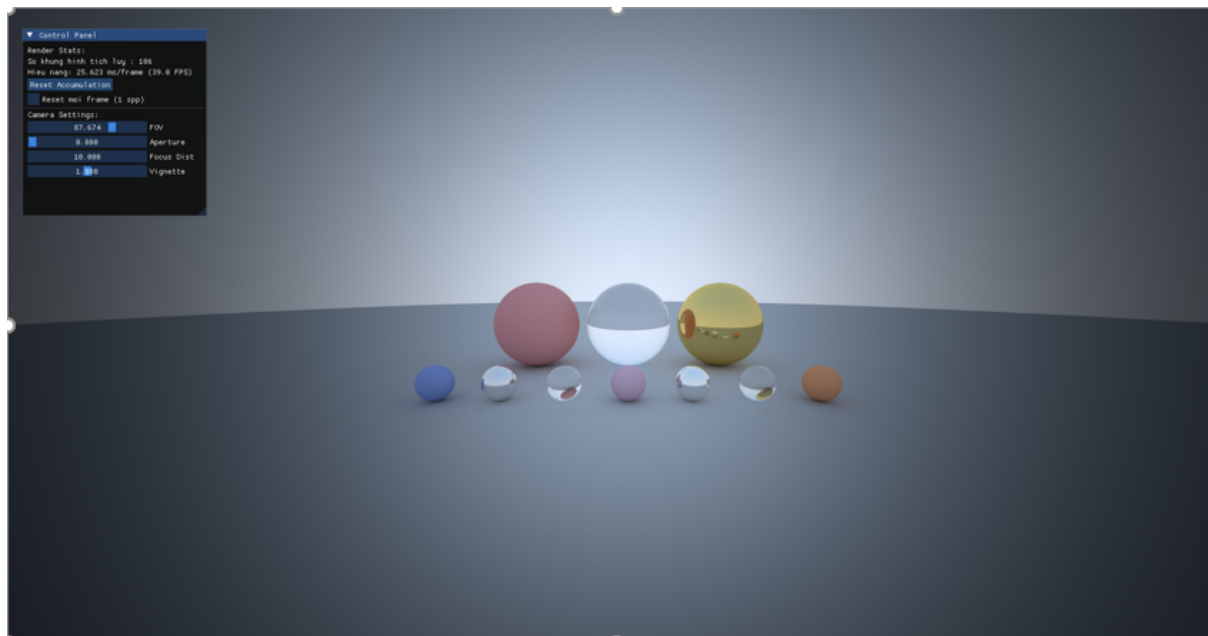
4.6 Làm tối rìa ảnh (Vignetting)

Hình 4.5 minh họa hiệu ứng làm tối rìa ảnh áp dụng lên cảnh nhiều vật liệu. Nguyên lý cốt lõi mà hiệu ứng này mô phỏng là định luật $\cos^4$ của quang học ống kính: độ rọi tại một điểm trên phim lệch góc θ so với trục quang giảm theo

$$E(\theta) = E_0 \cos^4 \theta, \quad (4.1)$$

trong đó E_0 là độ rọi tại tâm ảnh. Số mũ 4 đến từ tích của bốn thừa số cosine: khoảng cách từ thấu kính tới điểm ảnh xa hơn ở rìa đóng góp $\cos^2 \theta$ theo luật nghịch đảo bình phương, khẩu độ nhìn xiên nên diện tích chiếu giảm thêm $\cos \theta$, và bề mặt phim nghiêng so với tia tới góp nốt

$\cos \theta$. Vì $\cos \theta$ giảm nhanh khi ra xa trục, độ sáng tắt mạnh ở bốn góc trong khi tâm ảnh gần như không đổi. Hệ thống tái hiện quy luật này như một bước hậu xử lý điều khiển bằng tham số Vignette: giá trị càng lớn thì vùng tối ở rìa càng rõ và mở rộng vào trong, giúp hướng thị giác người xem vào chủ thể ở trung tâm.



Hình 4.5: Hiệu ứng làm tối rìa ảnh (vignetting): độ sáng giảm dần từ tâm ra các góc khung hình.

4.7 Nhận xét chung

Qua các thử nghiệm trên, hệ thống tái tạo đầy đủ những hiệu ứng quang học mục tiêu gồm phản xạ, khúc xạ, đổ bóng mềm, độ sâu trường ảnh và nhòe chuyển động, với chất lượng hình ảnh hội tụ tốt sau vài trăm đến vài nghìn khung hình tích lũy. Về hiệu năng, hệ thống duy trì tốc độ tương tác từ 10 đến 18 khung hình trên giây với cảnh thưa, nhưng giảm rõ rệt khi mật độ vật thể tăng cao do chi phí tìm giao điểm theo phương pháp duyệt tuyến tính. Kết quả này xác nhận tính đúng đắn của hệ thống đồng thời chỉ ra nút thắt hiệu năng cần được giải quyết bằng cấu trúc tăng tốc không gian, sẽ được bàn ở Chương 5.

Chương 5

Kết luận và hướng phát triển

5.1 Kết luận

Đề tài đã nghiên cứu và xây dựng thành công một bộ kết xuất Path Tracing chạy song song trên kiến trúc CUDA của NVIDIA, đạt được các mục tiêu đề ra ở Chương 1. Cụ thể, hệ thống đã:

- Hiện thực trọn vẹn thuật toán Path Tracing trên GPU bằng CUDA, với vòng dò đường theo kiểu lặp và cơ chế tích lũy mẫu theo thời gian để khử nhiễu Monte Carlo.
- Xây dựng hệ thống vật liệu Lambertian, kim loại và điện môi cùng mô hình camera vật lý hỗ trợ độ sâu trường ảnh và nhòe chuyển động.
- Hiển thị thời gian thực nhờ cầu nối CUDA–OpenGL, tránh chi phí sao chép dữ liệu ảnh qua lại giữa CPU và GPU.
- Nạp cảnh theo định dạng Mitsuba 3 và cung cấp giao diện điều khiển tương tác để tinh chỉnh camera cũng như theo dõi hiệu năng.

Kết quả thực nghiệm ở Chương 4 xác nhận hệ thống tái tạo đầy đủ các hiệu ứng quang học mục tiêu và duy trì tốc độ tương tác trên phần cứng phổ thông. Đồng thời, thực nghiệm cũng chỉ ra nút thắt hiệu năng khi mật độ vật thể tăng cao, do việc tìm giao điểm còn thực hiện bằng duyệt tuyến tính, mở ra các hướng phát triển tiếp theo.

5.2 Hướng phát triển

Trên nền tảng hiện có, hệ thống có thể được mở rộng theo các hướng sau:

1. **Bounding Volume Hierarchy (BVH):** xây dựng cấu trúc tăng tốc không gian để giảm chi phí tìm giao điểm từ tuyến tính xuống mức logarit, khắc phục nút thắt hiệu năng ở cảnh mật độ cao.
2. **1D/2D:** bổ sung các loại hình học và nguyên thủy một, hai chiều (đoạn thẳng, mặt phẳng, tam giác) bên cạnh khối cầu hiện tại.
3. **Texture Mapping:** hỗ trợ ánh xạ vân bề mặt để tăng độ chi tiết và chân thực cho vật liệu.
4. **Emission Material:** thêm vật liệu phát sáng để mô phỏng nguồn sáng vùng trực tiếp trong cảnh.

5. **Volume Rendering:** kết xuất môi trường tham gia (khói, sương, chất bán trong suốt) bằng mô hình tán xạ thể tích.
6. **Monte Carlo:** nâng cao chất lượng ước lượng Monte Carlo bằng lấy mẫu theo tầm quan trọng (Importance Sampling) và ước lượng sự kiện kế tiếp để giảm nhiễu nhanh hơn.

5.3 Mã nguồn

Toàn bộ mã nguồn của đề tài được công khai tại kho lưu trữ:

<https://github.com/KinhLupNe/Path-tracing-Render-CUDA>

Tài liệu tham khảo

- [1] Peter Shirley, Trevor David Black, and Steve Hollasch. *Ray Tracing in One Weekend*. The ray tracing in one weekend series edition, 2020.
- [2] Peter Shirley, Trevor David Black, and Steve Hollasch. *Ray Tracing: The Rest of Your Life*. The ray tracing in one weekend series edition, 2020.
- [3] Eric Veach. *Robust Monte Carlo Methods for Light Transport Simulation*. PhD thesis, Stanford University, 1997.
- [4] Matt Pharr, Wenzel Jakob, and Greg Humphreys. *Physically Based Rendering: From Theory to Implementation*. Morgan Kaufmann, 3rd edition, 2016.
- [5] James T. Kajiya. The rendering equation. *ACM SIGGRAPH Computer Graphics*, 20(4):143–150, 1986.
- [6] NVIDIA Corporation. Cuda c++ programming guide. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024.
- [7] Turner Whitted. An improved illumination model for shaded display. *Communications of the ACM*, 23(6):343–349, 1980.
- [8] Robert L. Cook, Thomas Porter, and Loren Carpenter. Distributed ray tracing. *ACM SIGGRAPH Computer Graphics*, 18(3):137–145, 1984.
- [9] Robert L. Cook and Kenneth E. Torrance. A reflectance model for computer graphics. *ACM Transactions on Graphics*, 1(1):7–24, 1982.
- [10] Christophe Schlick. An inexpensive brdf model for physically-based rendering. *Computer Graphics Forum*, 13(3):233–246, 1994.
- [11] Tomas Möller and Ben Trumbore. Fast, minimum storage ray-triangle intersection. *Journal of Graphics Tools*, 2(1):21–28, 1997.
- [12] Ingo Wald. On fast construction of sah-based bounding volume hierarchies. In *IEEE Symposium on Interactive Ray Tracing*, pages 33–40, 2007.
- [13] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. Optix: A general purpose ray tracing engine. *ACM Transactions on Graphics*, 29(4):1–13, 2010.

- [14] David B. Kirk and Wen-mei W. Hwu. *Programming Massively Parallel Processors: A Hands-on Approach*. Morgan Kaufmann, 3rd edition, 2016.
- [15] Andrew S. Glassner. *An Introduction to Ray Tracing*. Academic Press, 1989.
- [16] Wenzel Jakob, Sébastien Speierer, et al. Mitsuba 3: A retargetable forward and inverse renderer. <https://www.mitsuba-renderer.org/>, 2022.